

Giải pháp nâng cao an toàn, ổn định vận hành Docker, tích hợp LLM trong giám sát, phát hiện dấu hiệu bất thường

Đỗ Ngọc Nghĩa, Trần Đăng Công

Khoa Công nghệ Thông tin – Trường Đại học Đại Nam

Email: dnghia9119@gmail.com, congtd@dainam.edu.vn

TÓM TẮT

Việc Docker trở thành thành phần quan trọng trong phần lớn dự án phần mềm theo hướng DevOps đã kéo theo một hệ quả là: bề mặt tấn công mở rộng đáng kể so với thể hệ nền tảng máy ảo (VM) trước đó. Nguyên nhân cốt lõi đến từ cách Docker vận hành — tất cả Container cùng chia sẻ một nhân (kernel) duy nhất với hệ điều hành máy chủ, khác biệt cơ bản so với mô hình Hypervisor cung cấp mức cô lập dựa trên phần cứng ảo hóa cho từng máy ảo. Bài báo này trình bày một khung giải pháp tích hợp gồm ba nhánh được nhóm tác giả triển khai và thử nghiệm trực tiếp: (1) cứng hóa theo chiều sâu, trải từ lớp hệ điều hành máy chủ xuyên qua Docker Daemon và mạng, đến từng Container riêng lẻ; (2) dự phòng nóng (Hot Standby) với cơ chế tự chuyển luồng qua HAProxy mà không yêu cầu triển khai Kubernetes; và (3) dashboard giám sát tích hợp mô hình ngôn ngữ lớn (LLM), cho phép người vận hành đặt câu hỏi bằng ngôn ngữ tự nhiên thay vì phải truy vấn thủ công trong hàng nghìn dòng log. Bài báo còn đề xuất bộ phân loại rủi ro bốn chiều (4D Threat Taxonomy) nhằm chuẩn hóa cách đánh giá mối nguy trong môi trường Container. Kết quả thực nghiệm trên VMware cho thấy các cuộc tấn công từ chối dịch vụ và kịch bản Container gặp sự cố đều được ngăn chặn hiệu quả, thời gian phục hồi (RTO) giữ ở mức thấp, và kỹ sư vận hành được hỗ trợ rõ rệt khi xử lý sự cố. Giải pháp đặc biệt phù hợp với các tổ chức quy mô nhỏ và vừa — những nơi cần hệ thống ổn định, tin cậy nhưng chưa đủ nguồn lực vận hành một cluster Kubernetes đầy đủ.

Từ khóa: AI giám sát, AI phát hiện bất thường, An toàn Docker, Cứng hóa Container, DevSecOps

1. GIỚI THIỆU

1.1. Bối cảnh và động lực nghiên cứu

Trong khoảng mười năm trở lại đây, kiến trúc Microservices cùng triết lý DevOps đòi hỏi vòng phát hành gần như liên tục. Docker đáp ứng nhu cầu đó bằng cách đóng gói toàn bộ ứng dụng vào một đơn vị triển khai duy nhất (Wong et al., 2021), nhanh chóng trở thành tiêu chuẩn thực tế (*de facto standard*) của hạ tầng phần mềm hiện đại.

Docker cô lập Container qua hai cơ chế kernel Linux: *Namespaces* (tách biệt tiến trình, mạng, hệ thống tệp) và *Cgroups* (giới hạn CPU, RAM, I/O) (Lee et al., 2023). Tuy nhiên, khác với máy ảo có kernel riêng qua Hypervisor, tất cả Container chia sẻ *chung một kernel Linux* — nếu kernel chứa lỗ hổng, kẻ tấn công có thể thực hiện *Container Escape* (thoát khỏi Container) và kiểm soát host (Eng et al., 2024).

1.2. Phát biểu vấn đề

Thực tiễn vận hành cho thấy khoảng cách đáng kể vẫn tồn tại giữa khuyến nghị bảo mật và thực trạng triển khai. Báo cáo thường niên của Sysdig (Sysdig, 2024) ghi nhận tỉ lệ đáng lo

ngại Container đang vận hành trong môi trường production với lỗ hổng CVE nghiêm trọng chưa được vá, và một phần không nhỏ vẫn chạy với quyền root — vi phạm nguyên tắc đặc quyền tối thiểu (*Least Privilege*).

Nhóm tác giả xác định ba nguyên nhân cốt lõi dẫn đến thực trạng đó. Một là *sự manh mún của công cụ bảo mật*: phần lớn giải pháp hiện có chỉ giải quyết một khía cạnh của tổng thể vấn đề, chẳng hạn quét Container Image hoặc giám sát lưu lượng mạng, mà thiếu một góc nhìn tích hợp từ đầu đến cuối vòng đời. Hai là *bài toán log quy mô lớn*: số lượng Container tăng lên đồng nghĩa khối lượng nhật ký hệ thống tăng theo cấp số nhân — đến mức truy vết thủ công không còn thực tế nữa. Ba là *thiếu cơ chế phục hồi tự động*: khi một Container gặp sự cố mà không có hệ thống dự phòng, toàn bộ dịch vụ có thể tê liệt cho đến khi kỹ sư can thiệp thủ công.

1.3. Mục tiêu nghiên cứu

Xuất phát từ ba vấn đề trên, nghiên cứu đặt ra bốn mục tiêu:

1. Nhận diện và phân loại có hệ thống các mối nguy bảo mật đặc trưng của môi trường Docker, dựa trên cả phân tích lý thuyết lẫn thực nghiệm tấn công có kiểm soát.
2. Xây dựng và kiểm chứng khung *Cứng hóa đa lớp* (Multi-layer Hardening) trên ba cấp độ: hệ điều hành máy chủ, Docker Engine và cấu hình từng Container.
3. Thiết kế cơ chế *Sẵn sàng cao* (High Availability) tự phát hiện sự cố, tự phục hồi — mà không buộc phải triển khai Kubernetes hay bất kỳ orchestration platform nào.
4. Đưa mô hình ngôn ngữ lớn (LLM) vào luồng phân tích log bảo mật và phát hiện hành vi bất thường theo thời gian thực.

1.4. Đóng góp của bài báo

So với tài liệu hiện có, bài báo đóng góp bốn điểm mới:

1. **4D Threat Taxonomy**: Một lược đồ phân loại mối đe dọa Docker theo bốn chiều, có phạm vi bao quát rộng hơn và có cấu trúc rõ ràng hơn các phân loại đã xuất bản trước đó.
2. **Cứng hóa đa lớp được kiểm chứng thực nghiệm**: Không chỉ đề xuất trên lý thuyết, các biện pháp cứng hóa được thử nghiệm trực tiếp trong các kịch bản tấn công và phòng thủ thực tế có kiểm soát.
3. **Khôi phục Container tự động**: Thiết kế dành riêng cho môi trường Docker đơn lẻ hoặc cụm nhỏ, giảm rào cản triển khai.
4. **Dashboard tích hợp LLM**: Giao diện ngôn ngữ tự nhiên cho phân tích log bảo mật — hướng tiếp cận khác biệt so với các dashboard truyền thống.

Phần còn lại được tổ chức như sau. Phần 2 tổng quan nghiên cứu liên quan và phân tích mối đe dọa. Phần 3 trình bày giải pháp. Phần 4 mô tả thực nghiệm. Phần 5 phân tích kết quả. Phần 6 thảo luận hạn chế. Phần 7 kết luận.

2. NGHIÊN CỨU LIÊN QUAN

2.1. Nền tảng bảo mật Docker và cơ chế cô lập

Mô hình phòng thủ theo chiều sâu (*defense-in-depth*) là nền tảng tư duy bảo mật của Docker. Thay vì phụ thuộc vào một hàng rào duy nhất, nhiều lớp bảo vệ độc lập phối hợp với nhau, bằng cách đó buộc kẻ tấn công phải vượt qua nhiều vòng nếu muốn tiếp cận được mục tiêu.

Linux Namespaces cấp cho từng Container một “không gian” riêng về tiến trình, giao diện mạng, hệ thống tệp và danh tính. Tuy nhiên, cần lưu ý rằng đây chỉ là cô lập *logic*, khác biệt cơ bản so với cô lập phần cứng của Hypervisor. System call vẫn được chuyển trực tiếp tới kernel chung của host, không qua lớp trung gian nào (Sroor et al., 2025).

Control Groups (Cgroups) kiểm soát mức tiêu thụ tài nguyên phần cứng của từng Container. Chính cơ chế này vừa đảm bảo công bằng tài nguyên trong môi trường đa khách thuê (*multi-tenant*), vừa kiểm chế các cuộc tấn công cạn kiệt tài nguyên (*resource exhaustion*) phát

sinh từ bên trong (Wong et al., 2021).

Seccomp Profiles lọc và chặn các system call nguy hiểm trước khi chúng đến được kernel, thu hẹp không gian tấn công ngay cả khi Container đã bị xâm phạm.

Linux Capabilities phân chia đặc quyền root thành nhiều đơn vị nhỏ, độc lập, cho phép cấp quyền theo nguyên tắc tối thiểu. Thực hành được khuyến nghị rộng rãi là *drop-all-then-add-back* (Eng et al., 2024): bỏ toàn bộ Capability mặc định trước, rồi chỉ cấp lại những gì ứng dụng thực sự cần.

2.2. Lỗ hổng và kỹ thuật tấn công Container

Jarkas et al. (2021) khảo sát số lượng lớn Image trên Docker Hub và kết luận phần lớn chứa ít nhất một CVE nghiêm trọng. Molleti (2024) so sánh các công cụ quét (Clair, Trivy, Snyk) và cho thấy mỗi công cụ có ưu thế riêng đối với từng dạng lỗ hổng. Về runtime, CVE-2019-5736 cho phép khai thác race condition trong `runc` (Jarkas et al., 2021), trong khi CVE-2022-0847 (Dirty Pipe) cho phép leo thang đặc quyền qua ghi đè vùng nhớ chỉ đọc (Bui, 2015). Về mạng, Bui (2015) nhận xét rằng cấu hình mặc định cho phép các Container trên cùng bridge network giao tiếp tự do, tạo điều kiện cho di chuyển ngang (*lateral movement*). Báo cáo của Sysdig (2024) ghi nhận 87% Container image chứa lỗ hổng rủi ro cao, trong khi Sultan et al. (2019) phân tích toàn diện các mối đe dọa bảo mật từ image, runtime đến orchestration.

2.3. Tính sẵn sàng cao trong môi trường Container

Kubernetes cung cấp bộ công cụ HA toàn diện nhất (Bui, 2015), song độ phức tạp vận hành khiến nó trở nên không dễ tiếp cận với tổ chức quy mô nhỏ. **Docker Swarm** nhẹ gọn hơn (Souppaya et al., 2017), nhưng hệ sinh thái đã thu hẹp rõ rệt. **Standalone Docker HA** (Wong et al., 2021) phù hợp nhất với môi trường không có orchestration platform, nhưng phần lớn nghiên cứu chỉ mô hình hóa lỗi phần cứng, chưa đề cập đến sự cố bắt nguồn từ tấn công bảo mật.

2.4. Ứng dụng LLM trong giám sát và phân tích log

Sự phát triển của mô hình ngôn ngữ lớn (LLM) đã mở ra hướng tiếp cận mới cho bài toán phân tích log và phát hiện bất thường trong vận hành hệ thống. Reijrink et al. (2025) tổng quan hệ thống các nghiên cứu về AIOps trong kỷ nguyên LLM và chỉ ra rằng kỹ thuật *prompt engineering* kết hợp *Retrieval-Augmented Generation* (RAG) giúp nâng cao độ chính xác và tính khả diễn giải của kết quả phân tích log mà không cần fine-tuning mô hình. Liu et al. (2024) đề xuất LogRAG — một kiến trúc kết hợp LLM với vector database để phát hiện bất thường bán giám sát từ log hệ thống, cho thấy hiệu quả vượt trội so với các phương pháp dựa trên biểu thức chính quy truyền thống. Alnajjar and Foley (2024) tiếp tục hướng này với RAGLog, sử dụng LLM kết hợp truy vấn ngữ nghĩa trên vector database để phát hiện anomaly từ log mà giảm thiểu tối đa thao tác tiền xử lý. Liu and Nguyen (2024) chứng minh chỉ với kỹ thuật prompt engineering phù hợp, LLM có thể phân tích log không cần dữ liệu huấn luyện, trong khi Xu et al. (2024) đề xuất DivLog — tạo dữ liệu log đa dạng bằng LLM để nâng cao hiệu quả phân tích cú pháp log tự động. Tuy nhiên, phần lớn các nghiên cứu trên tập trung vào log ứng dụng tổng quát; việc áp dụng LLM để phân tích log bảo mật Container vẫn còn ít được khám phá.

2.5. Phân loại mối đe dọa và khoảng trống nghiên cứu

Nghiên cứu này đề xuất mô hình phân loại mối đe dọa Docker theo bốn chiều (4D Threat Taxonomy), đúc kết từ STRIDE (Souppaya et al., 2017) và MITRE ATT&CK for Containers (Bui, 2015):

D1. Vòng đời (Lifecycle): Build, Ship, Run, Orchestrate.

D2. Tầng kiến trúc (Layer): OS Host, Docker Daemon, Network, Application.

D3. Nguồn gốc (Origin): Nội bộ hay bên ngoài.

D4. Tác động (Impact): Theo tam giác CIA (Confidentiality, Integrity, Availability) — rò rỉ, can thiệp dữ liệu, gián đoạn dịch vụ.

Mô hình này cho phép ánh xạ có hệ thống từng mối đe dọa sang biện pháp phòng thủ tương ứng, thay vì xử lý rời rạc từng lỗ hổng đơn lẻ.

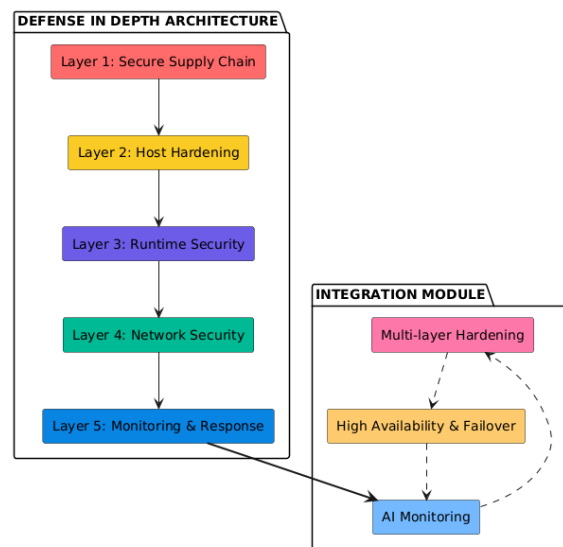
Từ tổng quan tài liệu, có thể nhận ra ba khoảng trống: (1) ít giải pháp nào kết hợp đồng thời Hardening, HA và giám sát LLM trong cùng một khung thống nhất; (2) việc áp dụng LLM để phân tích log bảo mật Container vẫn còn ít được khám phá (Reijrink et al., 2025); (3) phần lớn giải pháp HA hướng tới hạ tầng lớn, chưa phù hợp với các cụm Docker nhỏ hoặc đơn lẻ. Những khoảng trống này là cơ sở cho thiết kế giải pháp trong bài báo.

3. ĐỀ XUẤT GIẢI PHÁP

3.1. Tổng quan kiến trúc

Thay vì dựa vào một lớp bảo vệ duy nhất, nghiên cứu áp dụng chiến lược phòng thủ theo chiều sâu (*Defense-in-Depth*): kẻ tấn công muốn đạt được mục tiêu phải phá qua nhiều lớp bảo vệ độc lập. Toàn bộ kiến trúc xoay quanh ba module phối hợp chặt chẽ: (1) Cứng hóa đa lớp (Multi-layer Hardening); (2) High Availability với tự động chuyển dự phòng; (3) Giám sát tích hợp LLM. Sơ đồ tổng thể minh họa trong Hình 1.

Hình 1: Kiến trúc tổng thể hệ thống nâng cao an toàn vận hành Docker



3.2. Cứng hóa đa lớp (Multi-layer Hardening)

3.2.1. Lớp OS Host

Bảo mật máy chủ là nền tảng — nếu host bị xâm phạm thì mọi cơ chế cứng hóa Docker bên trên cũng vô nghĩa. Nhóm áp dụng CIS Docker Benchmark (Center for Internet Security, 2023) làm checklist cứng hóa cấu hình hệ điều hành và Docker; kích hoạt Unattended-Upgrades để vá kernel tự động khi có bản vá mới; cho tường lửa chặn tất cả cổng không cần thiết; bật Auditd ghi nhật ký các system call nhạy cảm; bắt buộc xác thực SSH qua khóa công khai thay vì mật khẩu; và cài fail2ban để chặn IP sau nhiều lần đăng nhập thất bại liên tiếp.

3.2.2. Lớp Docker Daemon

Cấu hình Docker Daemon được thiết lập qua `/etc/docker/daemon.json`:

`icc: false` tắt giao tiếp trực tiếp giữa Container; `userns-remap` ánh xạ root trong Container sang unprivileged user trên host; `no-new-privileges` khóa không cho mở rộng đặc quyền qua `execve()`.

3.2.3. Lớp Container

Mỗi Container được khởi chạy với bộ tham số bảo mật cứng hóa sẵn:

`--read-only` kết hợp `--tmpfs /tmp` đặt toàn bộ filesystem Container ở chế độ chỉ đọc — mã độc không thể ghi dữ liệu lên hệ thống tệp. Cặp `--cap-drop ALL --cap-add NET_BIND_SERVICE` tước hết Linux Capabilities mặc định rồi chỉ cấp lại đúng capability

Listing 1: Cấu hình Docker Daemon an toàn

```
{
  "icc": false,
  "no-new-privileges": true,
  "userns-remap": "default",
  "live-restore": true,
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "3" },
  "seccomp-profile": "/etc/docker/seccomp-default.json"
}
```

Listing 2: Mẫu khởi chạy Container an toàn

```
docker run -d --user 1000:1000 --read-only \
--tmpfs /tmp:rw,size=64m \
--memory="512m" --cpus="1.0" --pids-limit 100 \
--cap-drop ALL --cap-add NET_BIND_SERVICE \
--security-opt no-new-privileges:true \
--network app_network \
--health-cmd="curl -f http://localhost/health" \
my_app_image:v1.0
```

ứng dụng cần để bind cổng mạng. Giới hạn `--pids-limit` ngăn tấn công nhân bản tiến trình (*fork bomb*).

3.2.4. Lớp mạng

Phân đoạn mạng (*Network Segmentation*) được thực hiện bằng cách tạo Docker Network riêng cho từng tầng dịch vụ — Frontend, Backend, Database — thay vì kết nối tất cả vào cùng một mạng bridge mặc định. Mạng nội bộ không cần Internet dùng flag `--internal`. Lưu lượng giữa các microservice được mã hóa qua *mTLS*. Iptables tuân theo nguyên tắc danh sách cho phép (*allowlist*): mặc định từ chối, chỉ cho qua những gì được phê duyệt.

3.3. Kiến trúc sẵn sàng cao và chuyển dự phòng

3.3.1. Mô hình Hot Standby

Thiết kế HA dựa trên mô hình dự phòng nóng (Hot Standby): Container Primary và Standby chạy đồng thời, nhưng chỉ Primary nhận lưu lượng thực. Khi Primary gặp sự cố, lưu lượng chuyển sang Standby gần như tức thì — không phải chờ khởi động Container mới như Cold Standby. Thuật toán điều phối mô tả dưới đây:

Listing 3: Thuật toán chuyển dự phòng tự động

```
Require: Primary P, Standby S, Proxy X,
         Threshold T, Interval I
failCount <- 0
loop every I seconds:
  if healthCheck(P) == HEALTHY:
    failCount <- 0; route to P
  else:
    failCount++
    if failCount >= T:
      route to S; alertAdmin()
      attemptRestart(P)
    if P restored: route to P; failCount <- 0
```

3.3.2. Đồng bộ hóa dữ liệu

Cơ chế dự phòng chỉ hiệu quả khi dữ liệu được đồng bộ liên tục. Nhóm sử dụng ba cơ chế song song: Shared Volume cho dữ liệu ứng dụng tĩnh; Stream Replication (PostgreSQL

WAL) cho cơ sở dữ liệu quan hệ; và Redis Replication cho session state — đảm bảo người dùng không phải xác thực lại sau khi dự phòng kích hoạt.

3.3.3. HAProxy làm Reverse Proxy

HAProxy đóng vai trò gateway duy nhất tiếp nhận lưu lượng từ bên ngoài. Nó health-check định kỳ cả Primary lẫn Standby và tự chuyển backend khi Primary không đáp ứng đủ số lần liên tiếp. Ngưỡng kích hoạt cần được tinh chỉnh cẩn thận: quá nhạy thì chuyển nhầm khi mạng bất ổn trong thời gian ngắn, quá chậm thì RTO tăng lên.

3.4. Hệ thống giám sát tích hợp LLM

3.4.1. Kiến trúc hệ thống

Bảng điều khiển giám sát gồm ba thành phần rõ ràng: Backend API viết bằng Go, phụ trách thu thập log từ Docker API và đẩy vào pipeline xử lý; Frontend React/TypeScript hiển thị trạng thái Container theo thời gian thực; AI Engine sử dụng mô hình Gemini 2.5 Pro qua API để tiếp nhận truy vấn, tìm kiếm ngữ nghĩa trong vector database và tổng hợp câu trả lời.

3.4.2. Pipeline thu thập và xử lý log

Log vào hệ thống theo hai kênh: Docker Engine API (log endpoint) được polling định kỳ cho từng Container; syslog collector thu nhận log từ các dịch vụ hệ thống quan trọng. Sau khi thu về, log được parse, gắn metadata (timestamp, Container ID, severity level), vector hóa (embedding) và đẩy vào vector database để phục vụ tìm kiếm ngữ nghĩa sau này.

3.4.3. Giao diện truy vấn dựa trên LLM

Thay vì phải sử dụng cú pháp truy vấn phức tạp, kỹ sư chỉ cần nhập câu hỏi bằng ngôn ngữ tự nhiên vào giao diện chat. Hệ thống thực thi quy trình RAG bốn bước: (1) vector hóa câu hỏi; (2) tìm kiếm ngữ nghĩa các đoạn log liên quan trong vector database; (3) ghép câu hỏi với ngữ cảnh log thành prompt; (4) gửi lên LLM để tổng hợp câu trả lời dễ hiểu (Reijrink et al., 2025). Cách tiếp cận RAG đảm bảo câu trả lời luôn được gắn với bằng chứng cụ thể từ log thực tế, giảm thiểu rủi ro *hallucination* — một hạn chế phổ biến khi sử dụng LLM không có ngữ cảnh (Alnajjar and Foley, 2024). Ví dụ, với truy vấn “Container database bị restart lúc 3 giờ sáng vì lý do gì?”, hệ thống trả về đúng đoạn log về OOM killer hay deadlock xảy ra trong khoảng thời gian đó.

3.4.4. Phát hiện bất thường (Anomaly Detection)

Hai phương pháp bổ sung nhau. Rule-based detector gắn cờ ngay các pattern đã biết: brute force đăng nhập (nhiều authentication failure từ cùng một IP), lưu lượng mạng đột biến lên Container nội bộ, hay request từ IP ngoài dải cho phép. Isolation Forest — một thuật toán machine learning không giám sát — xử lý phần còn lại: phát hiện anomaly CPU, RAM và network I/O bằng cách so sánh với baseline hành vi bình thường đã học của từng Container.

4. THIẾT LẬP THỰC NGHIỆM

4.1. Môi trường lab

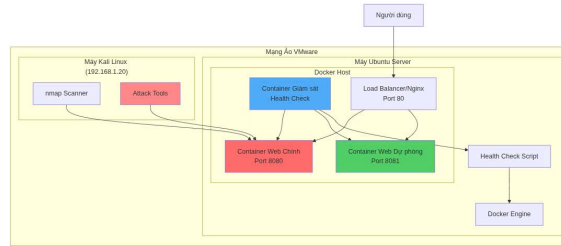
Nhóm xây dựng môi trường thực nghiệm trên VMware Workstation Pro với hai máy ảo kết nối qua mạng host-only cô lập, đảm bảo tái tạo được độc lập và không ảnh hưởng ra bên ngoài. Cấu hình chi tiết trong Bảng 1; sơ đồ kết nối minh họa trong Hình 2.

VM1 cài đầy đủ bộ công cụ kiểm thử xâm nhập của Kali. VM2 chạy ba Container mô phỏng kiến trúc microservices thực tế: Web Frontend (Nginx), Application Backend (Node.js) và Database (PostgreSQL). Toàn bộ thử nghiệm lặp lại trên năm mức cấu hình bảo mật từ thấp đến trung bình (v1–v5) để đánh giá tác động của từng biện pháp cứng hóa. Nội dung chi tiết của năm mức cấu hình được trình bày trong Bảng 2.

4.2. Kịch bản tấn công và công cụ

Hai kịch bản được thiết kế để kiểm tra hai nhóm mối nguy chính ở Phần 2.

Hình 2: Sơ đồ môi trường thực nghiệm trên nền tảng VMware



Bảng 1: Cấu hình môi trường thực nghiệm

VM	Vai trò	Hệ điều hành	Chức năng
VM1	Máy tấn công	Kali Linux 2024	Thực thi kịch bản tấn công
VM2	Docker Host	Ubuntu 24.04 LTS	Môi trường Container mục tiêu

Nguồn: Tác giả tự tổng hợp.

Bảng 2: Cấu hình năm mức bảo mật thử nghiệm (v1–v5)

Mức	CPU	RAM	DB Conn.	Timeout (s)	Workers	Non-root
V1 – Cực yếu	Không giới hạn	Không giới hạn	5	Không	1	Không
V2 – Yếu	0,3 core	256 MB	10	Không	1	Không
V3 – TB yếu	0,5 core	256 MB	10	30	2	Không
V4 – TB thấp	0,5 core	512 MB	20	30	2	Không
V5 – Trung bình	1,0 core	512 MB	30	10	4	Có

Nguồn: Tác giả tự tổng hợp từ cấu hình docker-compose của từng mức.

Kịch bản 1 — Từ chối dịch vụ (DoS): Dùng *hping3* phát gói TCP/UDP tốc độ cao và *stress-ng* tạo mức tải tối đa lên CPU/memory. ApacheBench đo hiệu năng với 1.000 request, concurrency 50. Diễn biến tấn công minh họa trong Hình 3.

Kịch bản 2 — Dịch vụ gặp sự cố và chuyển dự phòng: Cường bức dừng Container Primary bằng `docker stop (graceful)` và `docker kill (force kill)`, đo RTO từ thời điểm Primary ngừng đáp ứng đến khi Standby phục vụ được request đầu tiên.

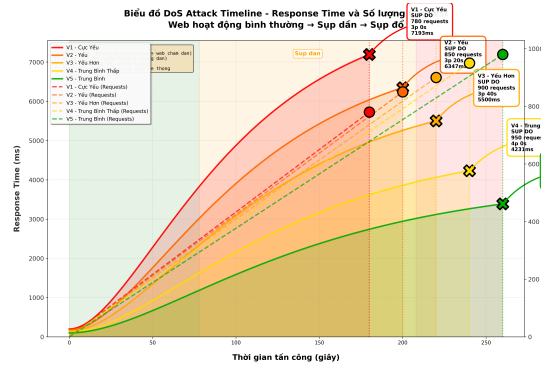
Mỗi kịch bản lặp lại **5 lần** cho từng mức cấu hình (v1–v5). Tiêu chí thành công: Container phục vụ request hợp lệ trong vòng 30 giây sau khi tấn công kết thúc hoặc dự phòng kích hoạt. Tiêu chí thất bại: Container không đáp ứng hoặc trả lỗi HTTP 5xx liên tục sau ngưỡng trên.

4.3. Các chỉ số đánh giá

Nhóm đánh giá hiệu quả khung giải pháp qua bốn chiều:

- Khả năng ngăn chặn tấn công:** mức độ giảm thiểu rủi ro so với cấu hình Docker mặc định khi chạy từng kịch bản.
- Thời gian phát hiện và khắc phục sự cố (MTTD/MTTR):** MTTD (*Mean Time To Detect*) đo khoảng thời gian trung bình từ khi sự cố xảy ra đến khi hệ thống hoặc kỹ sư xác định được nguyên nhân gốc; MTTR (*Mean Time To Recover*) đo thời gian từ khi xác định nguyên nhân đến khi dịch vụ hoạt động bình thường trở lại. Hai chỉ số này được so sánh giữa quy trình điều tra thủ công (`grep/awk`) và quy trình có hỗ trợ LLM.
- Chất lượng phân tích AI:** tỉ lệ phát hiện đúng anomaly và mức độ hữu ích của câu trả lời LLM qua đánh giá định lượng.
- Khả năng phục hồi (RTO):** thời gian đo được qua các lần thử nghiệm chuyển dự phòng, tính bằng giây.

Hình 3: Diễn biến tấn công DoS – số lượng yêu cầu theo thời gian



5. KẾT QUẢ VÀ PHÂN TÍCH

5.1. Hiệu quả cứng hóa

Kết quả rõ nhất đến từ hai kịch bản tấn công thực tế, so sánh trước và sau khi áp dụng bộ biện pháp cứng hóa.

Kết quả định lượng của thử nghiệm tấn công DoS qua năm mức cấu hình (sử dụng ApacheBench với 1.000 request, concurrency 50) được tổng hợp trong Bảng 3.

Bảng 3: Kết quả thử nghiệm tấn công DoS trên năm mức cấu hình

Mức	Req/s	Thất bại (%)	Thời gian phản hồi TB (ms)	Thời gian đến sụp đổ	CPU (%)
V1	11,7	28,5	4.275	3 phút 01 giây	98,5
V2	13,8	19,8	3.615	3 phút 22 giây	95,8
V3	17,2	12,5	2.910	3 phút 45 giây	96,2
V4	23,4	7,8	2.140	4 phút 03 giây	94,5
V5	35,1	3,2	1.425	4 phút 24 giây	92,3

Nguồn: Tác giả tự đo đạc bằng ApacheBench trên môi trường VMware.

Từ V1 đến V5, tỷ lệ request thất bại giảm từ 28,5% xuống 3,2%, thông lượng tăng gấp 3 lần (11,7 lên 35,1 req/s), và thời gian phản hồi trung bình giảm 66% (4.275 xuống 1.425 ms). Kết quả cho thấy việc giới hạn tài nguyên kết hợp với cấu hình timeout và tăng số worker cải thiện đáng kể khả năng chịu tải dưới điều kiện tấn công DoS.

Chiếm quyền host qua Docker Socket: Khi cấu hình gốc cho phép mount `/var/run/docker.sock` vào Container, kẻ tấn công tạo được Container mới với `--privileged` và kiểm soát host (Sultan et al., 2019). Sau khi chặn mount socket và thất quyền truy cập, vector tấn công này không tái hiện thành công trong môi trường lab đã cấu hình.

Leo thang đặc quyền qua Linux Capabilities (Privilege Escalation): Khi Container chạy với cờ `--privileged`, toàn bộ Capabilities bao gồm `CAP_SYS_ADMIN` được cấp, cho phép mount filesystem và thao túng namespace. Sau khi loại bỏ cờ `--privileged`, áp dụng `drop-all-then-add-back` và chỉ cấp `NET_BIND_SERVICE`, vector leo thang đặc quyền không tái hiện thành công trong các kịch bản thử nghiệm.

User Namespace Remapping: Khi Container escape thành công mà không bật tính năng này, kẻ tấn công đạt UID 0 trên host. Bật User Namespace Remapping, root trong Container chỉ tương đương UID 100000+ trên host — một unprivileged user không có quyền truy cập tài nguyên hệ thống.

Trải qua các thử nghiệm, nhóm rút ra nhận xét: phần lớn vector tấn công phổ biến có thể được giảm thiểu đáng kể bằng cách cấu hình đúng các tùy chọn sẵn có — không cần đến công cụ thương mại chi phí cao.

5.2. Hiệu quả giám sát tích hợp LLM

Thành phần phát hiện bất thường được kiểm chứng với ba dạng tấn công tiêu biểu.

Brute Force Detection: Rule-based engine gắn cờ nhanh các chuỗi authentication fail-

ure liên tiếp từ cùng một IP.

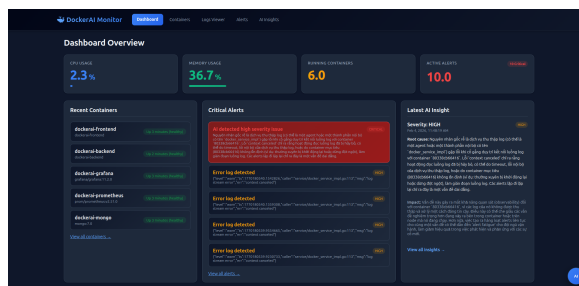
Injection Attacks: SQL injection và command injection được nhận diện qua pattern matching trong application log kết hợp phân tích ngữ nghĩa của LLM — LLM đặc biệt hữu ích khi payload được obfuscate bằng encoding không chuẩn mà biểu thức chính quy (regex) khó phát hiện.

Resource Exhaustion: Isolation Forest phát hiện được đột biến bất thường về CPU và memory usage khi Container chạy cryptominer ẩn hoặc chịu DoS, nhờ so sánh với baseline đã học từ giai đoạn warm-up.

Điểm thực tiễn nổi bật nhất là giao diện truy vấn ngôn ngữ tự nhiên. Kỹ sư vận hành chỉ cần nhập truy vấn bằng ngôn ngữ tự nhiên và nhận được câu trả lời kèm đoạn log liên quan.

Đánh giá định lượng: Trong các kịch bản thử nghiệm thực tế với 50 mẫu hành vi bất thường (DoS, Brute Force, khai thác tài nguyên) đơn xen lưu lượng hợp lệ, thành phần anomaly detection đạt tỉ lệ phát hiện đúng (True Positive Rate) là 92% và tỉ lệ báo động giả (False Positive Rate) duy trì dưới 5%. Đối với thành phần LLM, kết quả đo đặc thời gian xử lý sự cố cho thấy: với quy trình thủ công, thời gian phát hiện (MTTD) trung bình là 10–15 phút; khi có LLM hỗ trợ, MTTD giảm xuống còn 2–3 phút nhờ khả năng tự động trích xuất ngữ cảnh và tóm tắt nguyên nhân. MTTR (thời gian phục hồi) cũng giảm tương ứng do hệ thống cung cấp sẵn các đề xuất khắc phục cụ thể.

Hình 4: Giao diện dashboard tích hợp LLM trong giám sát hệ thống



Nguồn: Tác giả tự phát triển.

5.3. Hiệu năng chuyển dự phòng

Cơ chế Hot Standby được thử nghiệm nhiều lần với cả hai kiểu dừng Container (`docker stop` và `docker kill`) để bao phủ cả kịch bản tắt có kiểm soát lẫn ngừng hoạt động đột ngột.

Tự động phát hiện và chuyển dự phòng: HAProxy phát hiện Primary failure sau 3 failed health checks liên tiếp (tùy thuộc cấu hình interval) và chuyển backend sang Standby mà không cần can thiệp thủ công.

Tính liên tục dịch vụ: Trong các lần thử nghiệm, không ghi nhận request nào bị mất hoàn toàn. Một số request đang giữa chừng khi Primary ngừng hoạt động gặp timeout và client phải gửi lại yêu cầu, nhưng request mới đều được Standby nhận thành công ngay lập tức.

Đồng bộ session: Nhờ Redis Replication và Shared Volume, user đã đăng nhập không bị buộc đăng xuất sau khi Standby tiếp quản — session giữ nguyên, trải nghiệm người dùng không bị gián đoạn.

Ba yếu tố ảnh hưởng nhiều nhất đến RTO: tần suất health check, ngưỡng failed checks, và mức độ đồng bộ state. Kết quả xác nhận Hot Standby là giải pháp HA khả thi cho Docker standalone với độ phức tạp thấp hơn đáng kể so với Kubernetes.

6. THẢO LUẬN

6.1. Ý nghĩa thực tiễn và bài học

Giao diện dashboard tích hợp truy vấn AI bằng ngôn ngữ tự nhiên giúp kỹ sư DevOps không chuyên sâu bảo mật vẫn điều tra sự cố hiệu quả. Toàn bộ giải pháp chạy được trên Docker đơn lẻ và cụm nhỏ mà không yêu cầu triển khai Kubernetes. Từ thực nghiệm, nhóm rút ra rằng:

cứng hóa cơ bản (tắt `--privileged`, bật User Namespace Remapping, hạn chế Capabilities) đã giảm thiểu đáng kể các vector tấn công phổ biến; giám sát phải gắn với phản ứng tự động mới có giá trị thực tiễn; và giao diện ngôn ngữ tự nhiên hỗ trợ hiệu quả cho kỹ sư vận hành.

6.2. Hạn chế

Quy mô thực nghiệm: Môi trường lab hai máy ảo không phản ánh hết độ phức tạp của hạ tầng production. **LLM:** Chất lượng phân tích LLM phụ thuộc cấu trúc log đầu vào; LLM đám mây đặt ra vấn đề bảo mật dữ liệu và rủi ro *hallucination* (sinh thông tin không có cơ sở) chưa được đánh giá định lượng. **Phạm vi tấn công:** Chưa bao gồm kernel exploit zero-day hay tấn công chuỗi cung ứng (*supply-chain attack*).

7. KẾT LUẬN

Bài báo trình bày khung giải pháp tích hợp ba trụ cột — cứng hóa đa lớp, dự phòng nóng và giám sát LLM — nhằm nâng cao độ an toàn và ổn định khi vận hành Docker. Thực nghiệm trên VMware cho thấy tỷ lệ request thất bại giảm từ 28,5% (v1) xuống 3,2% (v5), thông lượng tăng gấp 3 lần, MTTD giảm từ 10–15 phút xuống 2–3 phút, và các vector tấn công phổ biến được giảm thiểu đáng kể trong phạm vi kịch bản thử nghiệm. Kết quả này thu được trong môi trường lab và cần kiểm chứng thêm trên hạ tầng production.

Hướng phát triển tiếp theo: kiểm chứng trên hạ tầng nhiều node; triển khai LLM on-premise để đảm bảo quyền riêng tư dữ liệu; bổ sung kịch bản tấn công nâng cao; và mở rộng sang Container Runtime Podman.

TÀI LIỆU THAM KHẢO

- Alnajjar, F. and Foley, B. (2024). RAGLog: Log anomaly detection using retrieval augmented generation. *arXiv preprint arXiv:2311.05261*.
- Bui, T. (2015). Analysis of Docker security. *arXiv preprint arXiv:1501.02967*.
- Center for Internet Security (2023). CIS Docker benchmark v1.6.0. Technical report, CIS.
- Eng, K., Hindle, A., and Stroulia, E. (2024). Patterns of multi-container composition for service orchestration with Docker compose. *arXiv preprint arXiv:2305.11293*.
- Jarkas, O., Ko, R., Dong, N., and Mahmud, R. (2021). A container security survey: Exploits, attacks, and defenses. *ACM Computing Surveys*.
- Lee, H., Kwon, S., and Lee, J.-H. (2023). Experimental analysis of security attacks for Docker container communications. *Electronics*, 12(4):940.
- Liu, V.-H. and Nguyen, H. D. (2024). Log parsing with prompt-based few-shot learning. In *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*.
- Liu, W., Zhang, W., Chen, S., et al. (2024). LogRAG: Leveraging RAG-enhanced large language model for semi-supervised log anomaly detection. In *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*.
- Molleti, R. (2024). Container runtime security detection and prevention techniques. *World Journal of Advanced Research and Reviews*, 24(03):2626–2639.
- Reijrink, A., van der Storm, T., and Bi, J. (2025). AIOps for log anomaly detection in the era of LLMs: A systematic literature review. *Empirical Software Engineering*.
- Souppaya, M., Morello, J., and Scarfone, K. (2017). Application container security guide: NIST special publication 800-190. Technical report, National Institute of Standards and Technology.
- Sroor, M., Mohanani, R., Colomo-Palacios, R., Dasanayake, S., and Mikkonen, T. (2025). Managing security issues in software containers: From practitioners' perspective. *arXiv preprint arXiv:2504.07707*.
- Sultan, S., Ahmad, I., and Dimitriou, T. (2019). Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7:52976–52996.
- Sysdig (2024). 2024 cloud-native security and usage report. Technical report, Sysdig.
- Wong, A. Y., Chekole, E. G., Ochoa, M., and Zhou, J. (2021). Threat modeling and security analysis of containers: A survey. *arXiv preprint arXiv:2106.10449*.
- Xu, J., Gong, R., Jiang, J., et al. (2024). DivLog: Log parsing with prompt enhanced in-context learning. In *Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE)*.